

LLM-Powered Knowledge Graph Extractor from Unstructured Text

Author Name

Department of Computer Science
Graduate Program – CS585 Knowledge Engineering
`author.email@university.edu`

Abstract

The *Knowledge Acquisition Bottleneck* – the slow, expensive, expert-driven process of structuring human knowledge for use in computational systems – has been a defining challenge of Knowledge Engineering for decades. Recent advances in Large Language Models (LLMs) suggest a path around this bottleneck by treating extraction as a zero-shot generation task. We present a web-based system that accepts arbitrary unstructured text and produces an interactive knowledge graph rendered live in the browser. The system uses a provider-agnostic adapter architecture supporting Anthropic Claude, OpenAI GPT, Google Gemini, and local models via Ollama, and enforces a hybrid knowledge model: open entity types combined with a controlled vocabulary of fifteen relations. Each extracted entity carries verbatim source sentences as provenance, and the graph can be exported as JSON or RDF/Turtle for downstream Semantic Web use. We describe the architecture, the prompt and schema design, and a qualitative comparison of extraction behaviour across providers on three sample texts.

Keywords: knowledge graphs · knowledge acquisition · large language models · information extraction · semantic web.

1 Introduction

Building rich, queryable knowledge bases has historically required a labour-intensive cycle of expert interviews, ontology design, and manual rule authoring – a process so slow and brittle that Feigenbaum’s original characterization of the *knowledge acquisition bottleneck* [1] continues to describe industry practice over forty years later. Tools such as Protégé and CycL ease modeling but do not eliminate the fundamental cost: a human still has to read every document, identify every relevant entity, and write down every relationship.

Large Language Models change the economics of this process. Trained on internet-scale text, modern LLMs can identify entities and relations in arbitrary text without task-specific training data, and can be steered toward structured output via tool-use APIs, JSON-schema-guided decoding, or constrained generation. The natural question is whether this capability can be wired into a usable application that takes the role traditionally played by a knowledge engineer: read text, identify what matters, structure it, and present it as a graph.

This paper describes such a system. It is a practical development project rather than a research contribution: the goal is to integrate the current state of the art into a self-contained, reproducible web application and to evaluate it qualitatively on representative inputs. The system supports four LLM providers behind a unified extraction interface, renders the resulting graph interactively using Cytoscape.js, and exports to RDF/Turtle for interoperability with the wider Semantic Web ecosystem. Source sentences are preserved alongside every extracted entity and edge, providing the kind of *provenance trail* that lets a human user verify or correct the model’s claims.

The remainder of the paper is organized as follows. Section 2 reviews related work in automated knowledge graph construction. Section 3 describes the system architecture. Section 4 discusses

the central design decisions: the knowledge model, the prompt, and the structured-output mechanism. Section 5 reports a qualitative comparative evaluation across providers. Section 6 discusses limitations and lessons learned. Section 7 concludes.

2 Related Work

Automated knowledge graph construction from text has evolved through three broad eras.

Rule-based and supervised pipelines. Early systems combined Named Entity Recognition (NER) with Relation Extraction (RE) models trained on annotated corpora. Mitchell et al.’s NELL system [2] demonstrated semi-automated knowledge base construction at scale but required ongoing human curation and was sensitive to domain shift. Such pipelines were brittle: a model trained on news text typically performed poorly on scientific or legal documents, and adding a new relation type meant collecting and labelling new training data.

Knowledge graph embeddings. A parallel line of work produced low-dimensional vector representations of entities and relations. TransE [3] introduced the principle that the embedding of a head entity plus the embedding of a relation should approximate the embedding of the tail; subsequent models including RotatE and ComplEx generalized this to richer geometric operations. These methods are oriented toward *reasoning over* an existing knowledge graph – predicting missing links, identifying inconsistencies – rather than constructing one from raw text.

LLM-driven extraction. Recent work demonstrates that LLMs serve as effective zero-shot or few-shot knowledge extractors, dramatically lowering the entry cost. Pan et al. [4] survey the converging space of LLMs and knowledge graphs and argue for a complementary relationship: LLMs supply natural-language understanding, knowledge graphs supply structured, queryable, verifiable facts. Microsoft’s GraphRAG framework [5] couples LLM-based extraction with graph-aware retrieval to improve question answering over document collections, and tools such as Neo4j’s *LLM Graph Builder* [6] have productized the same idea in industry. The system presented here adopts this third-era approach but emphasizes provider-agnostic operation, a controlled relation vocabulary, and per-entity provenance for academic and educational use.

3 System Architecture

The system is a small monorepo with two top-level components: a stateless Express backend that exposes the extraction API, and a Next.js frontend that handles input, visualization, and export. They communicate over a JSON HTTP interface; the frontend proxies `/api/*` to the backend during development. Figure 1 illustrates the data flow.



Figure 1: High-level data flow. The frontend never speaks to LLM providers directly; the backend mediates and normalizes responses to a single `{nodes, edges}` schema regardless of which provider produced them.

3.1 Provider Adapters

Each supported LLM is wrapped in an adapter exposing the same interface: `isConfigured()` and `extract(text, model)`. The four current adapters use the most reliable structured-output mechanism each provider offers:

- **Anthropic Claude** – a single forced tool call (`tool_choice: {type: "tool", name: "emit_graph"}`) whose `input_schema` is the graph JSON Schema. The model cannot return free text; the adapter unwraps the `tool_use` block and validates.
- **OpenAI GPT** – structured outputs via `response_format: {type: "json_schema", strict: true}`, which guarantees adherence at the API level.
- **Google Gemini** – `responseSchema` on `generationConfig` with `responseMimeType: "application/json"`.
- **Ollama** – the local-model adapter uses `format: "json"` on the `/api/chat` endpoint and applies post-hoc schema validation, since open models do not yet enforce schema-level constraints as strictly as commercial APIs.

A central registry auto-detects which adapters are configured (by checking environment variables for the commercial APIs and pinging `localhost:11434` for Ollama) and exposes only those to the UI. This keeps the application functional even when the user has only one provider available.

3.2 Endpoints

The backend exposes three endpoints:

- `GET /api/providers` – returns the list of currently usable providers and their model menus.
- `POST /api/extract` – the main call; accepts `{text, provider, model}` and returns `{nodes, edges, meta}`.
- `POST /api/export/turtle` – accepts a graph object and returns an RDF/Turtle serialization for download.

3.3 Frontend

The frontend is a single Next.js page divided into an input column and a graph column. Cytoscape.js renders the graph using a force-directed layout, with node colours mapped from the open-vocabulary entity type to a small fixed palette (Person, Organization, Location, Concept, Event, plus a default for everything else). Clicking a node opens a side panel showing the entity's description, the verbatim source sentences from which it was extracted, and lists of incoming and outgoing relations. Two export buttons download the graph as JSON or as Turtle.

4 Knowledge Model and Prompt Design

The most consequential design decisions in a system of this kind are the knowledge model – what counts as an entity, what counts as a relation – and the prompt that elicits it. We discuss each in turn.

4.1 Hybrid Schema: Open Types, Controlled Relations

A knowledge model can be *closed*, fixing both entity types and relation labels in advance, or *open*, letting the extractor invent them. Closed models produce clean, queryable graphs but force the system into a single domain; open models handle arbitrary text but tend to produce label drift – the same relation gets called *works_at*, *employed_by*, and *is_employee_of* across different runs.

We adopt a *hybrid*: entity types are open (the LLM picks the most natural type for each entity), but relation labels are drawn from a controlled vocabulary of fifteen items. Open entity types reflect the real-world fact that text contains an unbounded variety of entity kinds (Person, Theory, Product, Field of study, etc.), while controlled relations preserve graph queryability. This split is consistent with the recommendations of Pan et al. [4], who frame LLMs and knowledge graphs as complementary rather than adversarial.

The relation vocabulary, chosen to cover the relations most commonly observed in encyclopedic and biographical text, is:

```
works_for, member_of, founded_by, located_in, headquartered_in, part_of,  
created_by, developed_by, causes, influences, occurred_at, occurred_on,  
has_property, related_to, opposes.
```

The model is instructed to fall back to the deliberately-weak `related_to` when no other label fits, and to prefer the weaker `influences` over the stronger `causes` when causality is implied rather than stated. A post-extraction normalizer enforces the vocabulary: any out-of-vocabulary relation returned by the model is coerced to `related_to` and edges referencing nonexistent nodes are dropped.

4.2 Output Schema

The extraction target is a JSON object with two arrays. Listing 1 shows the schema in abbreviated form.

Listing 1: Output schema (abbreviated). Each node carries its source sentences, and each edge carries the verbatim sentence supporting it.

```
{  
  "nodes": [{  
    "id": "albert_einstein",  
    "label": "Albert Einstein",  
    "type": "Person",  
    "description": "German-born theoretical physicist.",  
    "source_sentences": ["Einstein was born in Ulm in 1879."]   
  }],  
  "edges": [{  
    "source": "albert_einstein",  
    "target": "general_relativity",  
    "relation": "developed_by",  
    "evidence": "Einstein developed the theory of general relativity in 1915."   
  }]   
}
```

Two design choices are worth highlighting. First, every node carries its *source sentences* (one to three verbatim quotes from the input where the entity appears) and every edge carries an evidence string. This adds output tokens but yields two important benefits: it lets the user verify the extraction by clicking through to evidence, and it discourages hallucination – a model asked to cite the supporting sentence is less likely to invent unsupported relations. Second, the `id` field uses `lowercase_with_underscores`, which doubles as a stable URI fragment in the RDF/Turtle export (e.g. `kg:albert_einstein`).

4.3 System Prompt

The system prompt is provider-agnostic: the same prompt is used for all four providers, with only the structured-output mechanism differing per adapter. It contains seven numbered rules covering open entity types, the controlled relation vocabulary (with explicit fallback guidance), id formatting conventions, mandatory source-sentence inclusion, the no-invented-facts constraint, and an instruction to return only the JSON object. We found in initial development that explicitly numbering rules and stating the fallback (“*If no specific relation fits, use `related_to`*”) reduced both label drift and refusal-to-extract behaviour.

4.4 RDF/Turtle Export

For interoperability with the Semantic Web stack, the system serializes graphs to Turtle using two custom namespaces (`kg:` for entities and `rel:` for relations), `rdfs:label` and `rdfs:comment` for human-readable strings, and RDF reification (`rdf:Statement`) to attach evidence sentences to specific triples. This is a deliberately simple mapping: it preserves provenance, is human-readable, and can be loaded directly into triple stores such as GraphDB or Apache Jena.

5 Qualitative Evaluation

Note for the author: the numbers and observations in this section are illustrative placeholders, intended to show the structure of the evaluation. Replace them with results from your own runs of the three sample texts on at least two providers before submission. The schema for replacement is preserved verbatim below.

We evaluated the system qualitatively on three sample texts of contrasting character: a biographical paragraph about Marie Curie, a description of OpenAI as an organization, and a concept-heavy passage on the greenhouse effect. Each text was processed by two providers (Claude Sonnet 4.5 and Gemini 2.5 Flash), and the outputs were inspected manually for entity coverage, relation correctness, and provenance fidelity. The metrics are reported as counts and qualitative judgements rather than precision/recall against gold-standard annotations, in keeping with the project’s scope.

5.1 Sample 1: Marie Curie (biographical)

Table 1: Extraction summary for the Marie Curie passage (illustrative numbers).

Provider	Nodes	Edges	Latency	Notable behaviour
Claude Sonnet 4.5	9	12	~3.5 s	Captured both Nobel Prizes as separate Event nodes.
Gemini 2.5 Flash	8	10	~2.1 s	Merged the two prizes into one node; faster.

Both providers identified the central entities (Marie Curie, Pierre Curie, Sorbonne, Curie Institute, polonium, radium) and the canonical relations (`works_for`, `founded_by`, `developed_by`). The notable difference was granularity: Claude tended to split the 1903 Physics prize and the 1911 Chemistry prize into two nodes, which is arguably more faithful to the source; Gemini merged them into a single “Nobel Prize” node.

5.2 Sample 2: OpenAI (organizational)

The OpenAI text is dense with named individuals and organizational facts. Both providers correctly identified Sam Altman, Elon Musk, Greg Brockman, and Ilya Sutskever as founders, and used `founded_by` edges from the OpenAI node. Claude additionally captured the Microsoft investment using `related_to` (no specific `invested_in` relation exists in the vocabulary, so the fallback was triggered as designed), with the supporting sentence preserved as evidence. Gemini omitted the Microsoft relationship in this run.

5.3 Sample 3: Greenhouse effect (concept-heavy)

This passage contains few named entities but many causal relations. Both providers used `causes` to link *enhanced greenhouse effect* to *global warming* and `influences` for the weaker connection between *deforestation* and atmospheric CO₂. Claude additionally extracted the IPCC as an Organization node with a `founded_by` edge to the United Nations, while Gemini treated IPCC as a peer concept node. Neither model invented relations not supported by the text.

5.4 Cross-provider observations

Across the three samples, three observations recurred:

1. *Schema enforcement matters.* The Anthropic tool-use mechanism and OpenAI’s strict JSON schema both produced output that parsed without error in every run. Ollama with `format : "json"` returned syntactically valid JSON but occasionally emitted relations outside the controlled vocabulary, which the post-hoc normalizer coerced.
2. *Provenance discourages drift.* Requiring an `evidence` string per edge reduced the rate at which models invented unsupported relations – a finding consistent with general guidance on chain-of-thought and citation-prompting.
3. *Granularity differs across providers.* Claude tended toward finer-grained entity decomposition; Gemini toward coarser, faster output. Neither is uniformly better; the choice depends on downstream use.

6 Discussion and Limitations

The system addresses the knowledge acquisition bottleneck for a specific class of inputs – short, well-formed, English-language passages – and does so well enough to serve as a usable demonstration of the third-era approach. Several limitations bound this claim.

First, the system is single-pass: each extraction is independent, with no cross-document entity linking. A second submission of the Marie Curie text would produce an entity with id `marie_curie` disconnected from any prior such entity. Extending the system to a persistent graph database with entity resolution would be the natural next step.

Second, evaluation in this paper is qualitative. A more rigorous study would construct a small gold-standard annotation of three to five passages, compute precision and recall of extracted nodes and edges against the gold standard, and report inter-annotator agreement. We did not undertake this in the present work because the project scope is practical-development rather than empirical-research, but the system is structured to support such an evaluation: the JSON output format makes diffing against gold-standard annotations straightforward.

Third, the controlled relation vocabulary is fixed at fifteen items. This was a deliberate scope decision – it covers most encyclopedic relations and avoids the label-drift pathology of fully open vocabularies – but it means certain domain-specific relations (e.g. `invested_in` for finance, `regulates` for biology) collapse into the generic `related_to`. A production system would likely make the vocabulary user-editable per project.

Fourth, the system inherits the well-documented failure modes of its underlying LLMs: occasional hallucination, sensitivity to phrasing, and degradation on long inputs. The provenance mechanism (requiring evidence per edge) mitigates but does not eliminate these.

7 Conclusion

We have presented an LLM-powered knowledge graph extractor that takes arbitrary unstructured text and produces an interactive, exportable graph in seconds. The system is built around a hybrid

knowledge model (open entity types, controlled relations), an evidence-bearing output schema, and a provider-agnostic adapter architecture covering four LLM backends. It illustrates concretely how the knowledge acquisition bottleneck – the central challenge of classical knowledge engineering – is reshaped, though not eliminated, by recent advances in language modelling.

The system serves as a proof-of-concept rather than a production tool: cross-document entity linking, quantitative evaluation against gold-standard annotations, and a user-editable relation vocabulary are the three most natural extensions. Even in its present form, however, it produces graphs of usable quality on diverse inputs and exports cleanly into the wider Semantic Web stack via RDF/Turtle, demonstrating that the gap between unstructured prose and a queryable knowledge graph is now traversable in a single click.

References

- [1] Feigenbaum, E.A.: The art of artificial intelligence: themes and case studies of knowledge engineering. In: Proceedings of the 5th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1014–1029 (1977).
- [2] Mitchell, T., Cohen, W., Hruschka, E., Talukdar, P., Yang, B., Betteridge, J., Carlson, A., Dalvi, B., Gardner, M., Kisiel, B., et al.: Never-Ending Learning. Communications of the ACM **61**(5), 103–115 (2018).
- [3] Bordes, A., Usunier, N., García-Durán, A., Weston, J., Yakhnenko, O.: Translating Embeddings for Modeling Multi-relational Data. In: Advances in Neural Information Processing Systems (NeurIPS) (2013).
- [4] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X.: Unifying Large Language Models and Knowledge Graphs: A Roadmap. IEEE Transactions on Knowledge and Data Engineering (2024). <https://arxiv.org/abs/2306.08302>
- [5] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research (2024). <https://arxiv.org/abs/2404.16130>
- [6] Neo4j: LLM Graph Builder: Transforming Unstructured Data into Knowledge Graphs (2024). <https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/>